Supply Chain Credit Contagion — Full Technical Project Roadmap

> ***One-Sentence Summary:*** *Build a heterogeneous, multi-relational supply-chain graph* `G = (V, E, X, W)` *and use hybrid Graph Neural Networks (GCN + GAT + CuGNN + PI-GNN) to model credit contagion, predict network-adjusted Probability of Default (PD) and Expected Loss (EL), and deliver a GPU-accelerated interactive dashboard for risk managers.*

🎯 What Are We Building?

A **systemic risk analysis tool** that answers:

> *"If one company fails financially, which other companies will also become risky because they depend on it — and by how much?"*

Traditional credit-risk evaluation treats firms as **isolated entities** and misses the systemic risk embedded in production and trade networks. This project:

1. Constructs a **heterogeneous, multi-relational supply-chain property graph** for the automotive manufacturing and parts sector
2. Uses **Graph Neural Networks with message passing** to model how credit risk propagates through that network
3. Predicts **network-adjusted Probability of Default (PD)** and **Expected Loss (EL)** for every company
4. Delivers a **GPU-accelerated interactive dashboard** for non-technical risk managers

Why This Matters — The Contagion Effect

Prior empirical microdata shows supply-chain network contagion can amplify:

- **Expected Loss (EL)** by ~**5.2×**
- **Value at Risk (VaR)** by ~**6.7×**
- **Expected Shortfall (ES)** by ~**4.4×**

These are not theoretical — they are observed amplification factors from real supply-chain disruptions.

How Contagion Propagates — The Mechanism

Credit contagion in supply chains follows a cascading failure pattern analogous to epidemic spreading in networks:

1. **Trigger Event:** A firm experiences idiosyncratic distress (e.g., bankruptcy, sanctions, natural disaster)
2. **Upstream Propagation (Supply Shock):** Customers of the failed firm lose a supplier → production disruptions, increased costs, delayed deliveries
3. **Downstream Propagation (Demand Shock):** Suppliers of the failed firm lose a customer → revenue drops, trade-credit defaults, cash flow crises
4. **Trade-Credit Channel:** Firms that extended payment terms (net-30/60/90) to the failed firm face accounts receivable write-offs → liquidity squeeze
5. **Cascading Amplification:** Weakened firms become more likely to default themselves, triggering secondary waves through *their* supply chains
6. **Systemic Feedback Loop:** Market confidence deteriorates; lenders tighten credit for the entire sector, further amplifying distress

This cascade can be modeled mathematically using **SIIS epidemic dynamics** on the graph — where the "infection" is financial distress and the "transmission rate" depends on edge weights (trade volumes, dependency ratios).

🛠️ Tech Stack

Layer	Tool	Why
Language	**Python 3.10+**	Standard for ML and data work
Graph Building	**NetworkX**	Build and analyze company networks; compute centrality metrics
GNN Model	**PyTorch Geometric**	Industry-standard GNN library (GCN, GAT, GraphSAGE)
Financial Data	**yfinance / SEC EDGAR API**	Free financial data for companies
Data Handling	**Pandas, NumPy**	Clean and process data in tables
NLP / NER	**spaCy / Hugging Face**	Extract supplier–customer dyads from SEC filings
Backend API	**FastAPI (Python)**	Serve model results to the frontend
Frontend	**React + D3.js / Cosmograph (WebGL)**	Interactive dashboard and GPU-accelerated graph rendering
Database	**SQLite (start) → PostgreSQL / Graph DB (later)**	Store company and relationship data
Version Control	**GitHub**	Collaborate and track changes

> 🚀 *You do NOT need to know all of these on Day 1. You will learn them phase by phase.*

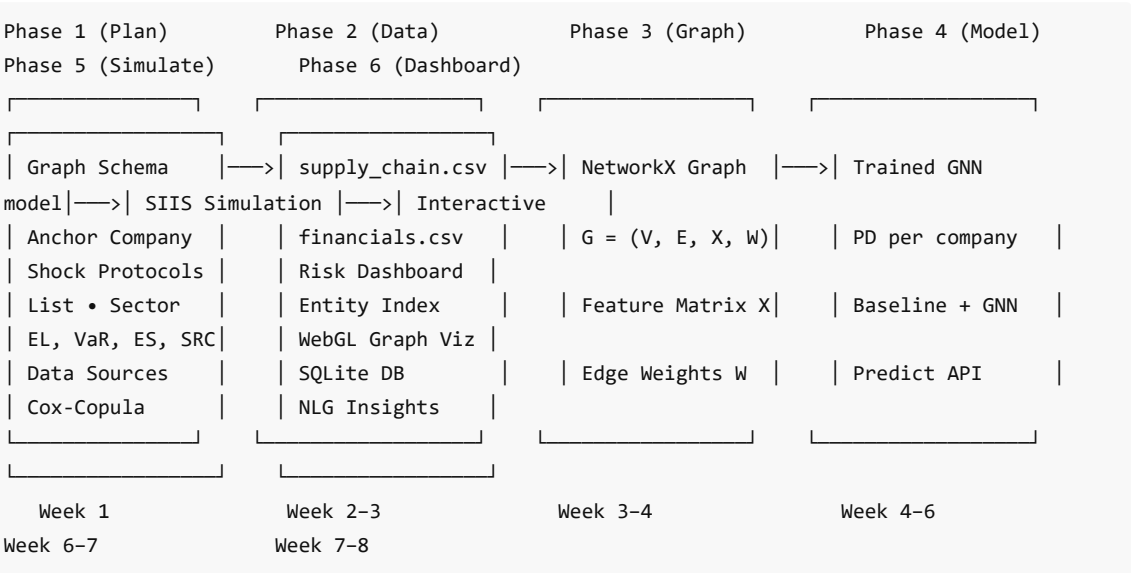
📁 Project Folder Structure

```
project/
├── data/              ← raw CSVs, cleaned datasets, SEC filings
│   ├── raw/           ← unprocessed source files
│   ├── processed/     ← cleaned, normalized data
│   └── external/      ← WIOD, Crunchbase, etc.
├── graph/             ← graph building scripts, entity resolution
│   ├── build_graph.py
│   ├── entity_resolution.py
│   └── network_metrics.py
├── model/             ← ML model code
│   ├── baseline/      ← logistic regression, random forest
│   ├── gnn/           ← GCN, GAT, GraphSAGE, PI-GNN
│   └── evaluation/    ← metrics, plots, reports
├── simulation/        ← contagion simulation engine
│   ├── siis_model.py  ← SIIS epidemic process
│   ├── shock.py       ← SCR-NR / SCR-ER shock protocols
│   └── stress_test.py
```

```
├── api/                ← FastAPI backend
├── frontend/           ← React dashboard
├── notebooks/          ← Jupyter notebooks for experiments
├── tests/              ← unit and integration tests
└── README.md
```

Phase-by-Phase Roadmap

How the phases connect — each phase produces artifacts consumed by the next:



Phase 1 — Setup & Planning (Week 1)

Goal: Get organized before writing a single line of code.

Setup Tasks:

- [ ] Create a shared GitHub repository
- [ ] Install Python 3.10+, VS Code, Git
- [ ] Agree on project scope: **~300 companies, ~1,000 supply relationships**
- [ ] Set up a weekly sync meeting (30 min/week minimum)
- [ ] Create the folder structure above in your repo

Technical Decisions to Make:

- [ ] Define the **graph schema** — your graph is `G = (V, E, X, W)` where:
  - `V` = corporate entities (nodes)
  - `E` = supply, trade-credit, and ownership linkages (edges)
  - `X` = node feature matrix (financial metrics + topological features)
  - `W` = edge weight matrix (relationship intensities)
- [ ] Choose your target sector (recommended: **automotive manufacturing & parts** — best data availability)

- [ ] List the 20–30 "anchor" companies you'll start with (Toyota, Ford, Apple, TSMC, etc.)

Formal Graph Schema Specification

The graph `G = (V, E, X, W)` is a **heterogeneous, directed, weighted, multi-relational property graph**:

Component	Mathematical Definition	Domain	Description
V (Nodes)	Set of `n` corporate entities	`	V
E (Edges)	Set of directed edges `(v_i, v_j, r)`	`	E
X (Features)	Node feature matrix `X ∈ ℝ^{n × d}`	`d ≈ 16`	Financial metrics + topological features per node
W (Weights)	Edge weight matrix `W_ij ∈ [0, 1]`	Normalized	Relationship intensity (trade volume, dependency ratio)
A (Adjacency)	`A ∈ {0,1}^{n × n}` with `A_ij = 1` if `(v_i, v_j) ∈ E`	Binary	Captures graph topology for GNN message passing
D (Degree)	Diagonal matrix `D_ii = Σ_j A_ij`	Integer	Node degree for Laplacian normalization
L (Laplacian)	`L = D - A` (unnormalized), `L_sym = D^{-1/2} L D^{-1/2}`	Real	Spectral basis for graph convolutions

Edge type semantics and directionality:

Edge Type	Direction	Risk Propagation	Example
`supply`	Supplier → Customer	Upstream failure → downstream disruption	Denso → Toyota
`trade_credit`	Creditor → Debtor	Default → accounts receivable write-off	Bosch → BMW (net-60 terms)
`ownership`	Parent → Subsidiary	Contagion via equity exposure	Toyota → Denso (24.7% stake)

Data Source Evaluation

Source	Data Type	Cost	Coverage	Priority
SEC EDGAR 10-K	Supplier/customer names (Items 1, 1A, 7)	Free	US public companies	🔴 Critical
yfinance API	Financial metrics (revenue, debt, ratios)	Free	Global public companies	🔴 Critical
Company Annual Reports	Financials for private firms (Bosch, ZF)	Free	Private companies	🔴 Critical
Kaggle	Pre-built supply chain datasets	Free	Variable	🟡 Supplementary

FactSet Revere	Curated supplier-customer relationships	Paid (trial)	Broad coverage	🟡 Nice-to-have
Panjiva / ImportGenius	Customs/shipment records → volume inference	Paid	Transaction-level	🟢 Stretch
WIOD	Macro inter-sector trade flows for shocks	Free	Sectoral aggregates	🟢 Phase 5
CSMAR	Auto supplier lists (Chinese market)	Institutional	Asia-Pacific	🟢 Supplementary

---

Phase 2 — Data Collection & Engineering (Week 2–3)

> ***Where you are in the pipeline:*** *Phase 1 finalized the graph schema* `G = (V, E, X, W)` *, chose the automotive sector, and listed 20–30 anchor companies. Now you will* ***populate*** *that schema with real data — extracting supplier–customer relationships from SEC filings, pulling financial metrics via APIs, and resolving entity names into a unified index. The outputs of this phase are two key CSVs (* `supply_chain.csv` *and* `company_financials.csv` *) and a master entity index — these become the raw ingredients for graph construction in Phase 3.*

Goal: Build your raw dataset of company relationships and financial data, then resolve entities into a unified graph.

Primary challenges: aggregating, aligning, and entity-resolving unstructured and semi-structured alternative data sources into a unified relational schema that supports deep graph learning at scale.

Data Engineering Specification Summary

Component / Task	Data Sources	Processing Methodology	Expected Deliverable
Regulatory Parsing	SEC 10-K/20-F filings, Annual Reports	HTML → Markdown pipelines; NLP NER for supplier–customer dyads	Structured edgelist of disclosed corporate dependencies
Transaction & Trade Mapping	Panjiva shipment data, CSMAR auto supplier lists	ML parsing of customs data; aggregate volume & frequency for edge weights	Weighted, directed edges representing physical trade intensity
Entity Resolution	SEC Metadata, Crunchbase, Panjiva entities	Probabilistic record linkage; graph-based entity alignment	Unified master entity index with UUIDs
Macroeconomic Anchoring	World Input-Output Database (WIOD)	Knowledge-graph alignment merging micro networks with macro flows	Exogenous risk features mapped to corporate nodes
Graph Schema Compilation	All processed sources	Instantiate heterogeneous graph with multiple edge types	Queryable multi-relational property graph

Data Collection Tasks

Task 2A: Supply Chain Data — Who Supplies Whom

Source 1 — SEC EDGAR (Primary, Free)

- Go to SEC EDGAR
- Search for large companies (Toyota, Ford, Apple, etc.)
- Open their **10-K annual report** → search for "major suppliers" or "significant suppliers"
- Extract supplier–customer dyads into a CSV

Technical Enhancement — NLP/NER Extraction:

Instead of only manual extraction, use **NLP Named Entity Recognition** to semi-automate this:

```
# --- NLP-based supplier extraction from SEC filings ---
# Step 1: Convert filed HTML to Markdown (preserves headings/tables, reduces tokens)
# Step 2: Apply NER to extract company names from supplier sections
# Step 3: Output structured edgelist of corporate dependencies

import spacy
nlp = spacy.load("en_core_web_sm")  # load English NER model

def extract_suppliers_from_filing(text):
    """
    Parse a 10-K filing text and extract organization names.
    Returns a list of detected company/organization names.
    """
    doc = nlp(text)
    # Filter entities labeled as ORG (organizations)
    suppliers = [ent.text for ent in doc.ents if ent.label_ == "ORG"]
    return list(set(suppliers))  # deduplicate
```

Resulting CSV format — `supply_chain.csv` :

```
supplier,company,relationship_type,data_source
SteelCorp,Toyota,tier1_supplier,sec_10k
ElectroParts,Toyota,tier1_supplier,sec_10k
SteelCorp,Ford,tier1_supplier,sec_10k
TSMC,Apple,tier1_supplier,sec_10k
```

Source 2 — Alternative / Supplementary Datasets:

- Kaggle Supply Chain datasets — search "supply chain network"
- FactSet Revere (paid, trial available)
- **Panjiva** shipment data (customs records) — parse with ML to infer shipment volumes
- **CSMAR** auto supplier lists
- **Federal Reserve Bank of New York / Crunchbase** dataset for cross-validation
- **World Input-Output Database (WIOD)** — macro sectoral flows for exogenous shock parameters

> 📦 *Start with a **manual toy network of 20–30 companies** to get code working before scaling to 300.*

Task 2B: Financial Data

Use `yfinance` in Python to pull financial metrics:

```
# --- Pull financial data for each company ---
import yfinance as yf
import pandas as pd

def get_financial_features(ticker):
    """
    Fetch key financial metrics for a given stock ticker.
    These become the intrinsic node features X_i in our graph.
    """
    stock = yf.Ticker(ticker)
    info = stock.info
    return {
        'company': info.get('shortName', ticker),
        'ticker': ticker,
        'revenue': info.get('totalRevenue', 0),        # total annual revenue
        'debt': info.get('totalDebt', 0),               # total debt outstanding
        'market_cap': info.get('marketCap', 0),          # market capitalization
        'current_ratio': info.get('currentRatio', 0),    # liquidity: current assets /
current liabilities
        'debt_to_equity': info.get('debtToEquity', 0),   # leverage ratio
        'profit_margin': info.get('profitMargins', 0),   # profitability
        'revenue_growth': info.get('revenueGrowth', 0),  # year-over-year growth
    }

# Example: fetch for multiple companies
tickers = ['TM', 'F', 'AAPL', 'TSM']  # Toyota, Ford, Apple, TSMC
financial_data = pd.DataFrame([get_financial_features(t) for t in tickers])
```

Default labeling:

- Mark companies that have historically defaulted → `default_label = 1`
- Healthy companies → `default_label = 0`
- For companies without clear default history, use **credit rating downgrades** (BBB- or below) as a proxy

Final financial dataset — `company_financials.csv` :

```
company,ticker,revenue,debt,market_cap,current_ratio,debt_to_equity,profit_margin,revenue_growt
Toyota,TM,280000000000,200000000000,250000000000,1.1,1.2,0.07,0.05,0
SteelCorp,STLC,4000000000,2000000000,3000000000,0.9,2.1,0.03,-0.02,0
ElectroParts,ELCP,1000000000,500000000,800000000,0.6,3.5,0.01,-0.10,1
```

Task 2C: Entity Resolution (Critical Step)

> ***Why this step matters:*** *You now have supplier names from SEC filings (Task 2A) and financial data from yfinance (Task 2B), but the same company may appear as "Toyota Motor Corp" in one source and "TOYOTA" in another. Before constructing the graph in Phase 3, all names must resolve to a single canonical entity. This step creates the* ***master entity index*** *— without it, the graph will contain duplicate nodes and broken edges.*

Problem: The same company appears differently across data sources (e.g., "Toyota Motor Corp", "Toyota Motor Corporation", "TOYOTA").

Solution — Multi-strategy entity resolution:

```
# --- Entity Resolution Pipeline ---
# Combines three strategies to build a master entity index with UUIDs

import uuid
from fuzzywuzzy import fuzz  # fuzzy string matching library

def resolve_entities(names_list):
    """
    Match company names across datasets using:
    1. Deterministic exact match (after normalization)
    2. Probabilistic string distance (fuzzy matching)
    3. Graph-based similarity (shared neighbors = likely same entity)
    Returns a dict mapping raw names → canonical UUID
    """
    master_index = {}
    canonical_names = []

    for name in names_list:
        normalized = name.strip().upper()  # normalize case and whitespace
        matched = False

        for canonical in canonical_names:
            # Fuzzy match: ratio > 85 means very likely the same entity
            if fuzz.ratio(normalized, canonical) > 85:
                master_index[name] = master_index[canonical]
                matched = True
                break

        if not matched:
            # New entity — assign a UUID
            entity_id = str(uuid.uuid4())
            master_index[name] = entity_id
            master_index[normalized] = entity_id
            canonical_names.append(normalized)

    return master_index
```

Infrastructure & API Tasks

> ***Parallel track:*** *While data collection and entity resolution are underway above, the following infrastructure tasks can proceed in parallel. The database schema and FastAPI skeleton built here will be* ***consumed by Phase 3*** *when the graph data needs to be served to the frontend, and by* ***Phase 4*** *when the model predictions need an API endpoint.*

- [ ] Set up **SQLite database** schema to store companies and relationships
- [ ] Write a Python script that reads CSVs and loads them into the database
- [ ] Install and test **FastAPI** with a simple "Hello World" endpoint:

```
# --- Minimal FastAPI setup ---
from fastapi import FastAPI
```

```
app = FastAPI(title="Supply Chain Contagion API")

@app.get("/")
def root():
    """Health check endpoint."""
    return {"status": "running", "message": "Supply Chain Contagion API"}

# Run with: uvicorn api.main:app --reload
```

- [ ] Create database schema:

```
-- companies table: stores node data
CREATE TABLE companies (
    id TEXT PRIMARY KEY,        -- UUID from entity resolution
    name TEXT NOT NULL,
    ticker TEXT,
    revenue REAL,
    debt REAL,
    market_cap REAL,
    current_ratio REAL,
    debt_to_equity REAL,
    profit_margin REAL,
    revenue_growth REAL,
    default_label INTEGER DEFAULT 0
);

-- relationships table: stores edge data
CREATE TABLE relationships (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    supplier_id TEXT,           -- FK to companies.id
    company_id TEXT,            -- FK to companies.id
    relationship_type TEXT,     -- 'tier1_supplier', 'trade_credit', 'ownership'
    weight REAL DEFAULT 1.0,    -- edge weight (transaction intensity)
    data_source TEXT,
    FOREIGN KEY (supplier_id) REFERENCES companies(id),
    FOREIGN KEY (company_id) REFERENCES companies(id)
);
```

Phase 3 — Graph Construction & Feature Engineering (Week 3–4)

> ***Where you are in the pipeline:*** *Phase 2 produced three key artifacts: (1)* `supply_chain.csv` *— an edgelist of who-supplies-whom, (2)* `company_financials.csv` *— financial metrics for each company, and (3) a master entity index mapping all name variations to canonical UUIDs. These CSVs are flat tabular data — they don't encode the rich network structure we need for contagion modeling. This phase* ***transforms*** *those flat tables into a live graph* `G = (V, E, X, W)` *and computes the node/edge features that will feed into the GNN in Phase 4.*

Goal: Turn CSV data into a real network graph `G = (V, E, X, W)` and compute features.

Feature Engineering Specification Summary

Feature Category	Specific Metrics	Mathematical Formulation	Purpose in Contagion Model
Intrinsic Node Attributes	Leverage, Liquidity, Revenue Growth, Baseline PD	Matrix $X_i \in \mathbb{R}^d$; extracted from normalized financials	Idiosyncratic vulnerability prior to network shock
Topological Node Features	Degree centrality, Eigenvector centrality, PageRank	$CE(i) = \lambda_1^{-1} \Sigma_{j \in N(i)} CE(j)$	Quantifies systemic importance & transmission potential
Local Density Metrics	Clustering coefficients	$C_i = 2e_i / (k_i(k_i - 1))$	Identifies redundancy vs. critical single points of failure
Edge Attributes	Transaction intensity, trade credit volume, dependency ratio	Asymmetric w_{ij} normalized by total flows	Conduit capacity and directional magnitude of risk propagation
Temporal Edge Features	Irregular time series encoding	tCNN processing for non-equidistant observations	Captures lead-time, seasonality, decay of relationships

Graph Building & Feature Computation

Task 3A: Build the Graph

```
# --- Build the supply chain graph ---
import networkx as nx
import pandas as pd

df = pd.read_csv('data/processed/supply_chain.csv')

# Create a directed graph: edge from supplier → company (A supplies B)
G = nx.DiGraph()

for _, row in df.iterrows():
    G.add_edge(
        row['supplier'],
        row['company'],
        relationship_type=row.get('relationship_type', 'supply'),
        weight=row.get('weight', 1.0)  # edge weight = transaction intensity
    )

print(f"Nodes: {G.number_of_nodes()}, Edges: {G.number_of_edges()}")
```

Task 3B: Compute Intrinsic Node Features $X \in \mathbb{R}^{|V| \times d}$

These are each company's **own** financial metrics — defining **idiosyncratic vulnerability** prior to network contagion:

Feature	Formula / Source	Purpose

Leverage	Debt-to-Equity ratio	How indebted is the firm?
Liquidity	Current Ratio, Quick Ratio	Can it pay short-term obligations?
Profitability	Profit margins	Is the firm operationally healthy?
Revenue Growth	YoY % change	Is the firm growing or shrinking?
Baseline PD	Credit score / rating proxy	Pre-network default probability

Task 3C: Compute Extrinsic Topological Features

These are **network-derived** features — quantifying each company's structural importance:

```
# --- Compute network metrics for each node ---
# These become additional features in the node feature matrix X

# Degree: how many connections does this company have?
in_degree = dict(G.in_degree())    # number of suppliers
out_degree = dict(G.out_degree())  # number of customers

# Betweenness Centrality: is this company a bridge between others?
# High betweenness = removing this node disconnects many paths
betweenness = nx.betweenness_centrality(G)

# Eigenvector Centrality: transitive influence
# A node is important if its neighbors are also important
eigenvector = nx.eigenvector_centrality(G, max_iter=1000)

# PageRank: how important is this company in the network?
# Similar to Google's algorithm — importance propagates through links
pagerank = nx.pagerank(G)

# Clustering Coefficient: local redundancy vs. bottleneck risk
# High clustering = many alternative paths; Low = single point of failure
clustering = nx.clustering(G.to_undirected())
```

Mathematical definitions:

Metric	Formula	Interpretation
Eigenvector Centrality	$CE(i) = \lambda_1^{-1} \Sigma_{j \in N(i)} CE(j)$	Importance based on neighbors' importance
Clustering Coefficient	$C_i = 2e_i / (k_i(k_i - 1))$	Fraction of possible triangles that exist
PageRank	Iterative $PR(i) = (1-d)/N + d \Sigma_{j \to i} PR(j)/L(j)$	Damped random-walk importance

Task 3D: Compute Edge Features `W`

Edge Feature	Description	How to Compute

Transaction intensity	Normalized shipment volumes	w_{ij} = volume_ij / total_volume_j
Trade-credit volume	Monetary exposure from disclosures	Extract from 10-K filing amounts
Supplier dependency ratio	Share of buyer's COGS from a given supplier	dep_ij = purchase_ij / total_COGS_j
Temporal decay	Recency weighting	Exponential decay on last transaction date

Task 3E: Temporal Feature Encoding

For irregular, non-equidistant transaction time series, use **temporal convolutional feature extractors (tCNN)**:

```
# --- Temporal encoding for edge features ---
# Distinguishes active vs. dormant relationships
# Captures seasonality and lead times

import numpy as np

def temporal_decay_weight(last_transaction_date, current_date, half_life_days=180):
    """
    Compute exponential decay weight based on recency.
    More recent transactions get higher weight.
    half_life_days: after this many days, weight drops to 0.5
    """
    days_elapsed = (current_date - last_transaction_date).days
    decay_rate = np.log(2) / half_life_days
    return np.exp(-decay_rate * days_elapsed)
```

API & Visualization Tasks

> ***Connecting to the frontend:*** *With the graph* `G` *built and all features computed, the next step is making this data accessible to a visual interface. The API endpoint below serializes the graph into JSON format that D3.js or vis.js can render. This is the first time you'll* ***see*** *your supply chain network as an interactive visualization — an essential feedback loop for catching data quality issues (orphan nodes, missing edges, unexpected clusters) before training the GNN in Phase 4.*

- [ ] Write API endpoint: `GET /graph` → returns all nodes and edges as JSON
- [ ] Use **D3.js** or **vis.js** in React to render a basic visual graph
- [ ] Each node = a circle (company), each edge = a line (relationship)
- [ ] Node size proportional to **betweenness centrality** (systemic importance)

```
# --- API endpoint to serve graph data ---
@app.get("/graph")
def get_graph():
    """
    Return full graph as JSON with nodes (+ features) and edges (+ weights).
    Frontend uses this to render the network visualization.
    """
    nodes = [
```

```
        {
            "id": node,
            "degree": G.degree(node),
            "betweenness": betweenness[node],
            "pagerank": pagerank[node],
            "clustering": clustering.get(node, 0)
        }
        for node in G.nodes()
    ]
    edges = [
        {"source": u, "target": v, "weight": d.get("weight", 1.0)}
        for u, v, d in G.edges(data=True)
    ]
    return {"nodes": nodes, "edges": edges}
```

Phase 4 — Machine Learning Model (Week 4–6)

> ***Where you are in the pipeline:*** *Phase 3 delivered a complete graph* `G = (V, E, X, W)` *— with nodes (companies), edges (supply/trade-credit/ownership links), node features* `X` *(financial metrics + topological centrality measures), and edge weights* `W` *(transaction intensities, dependency ratios). You also have a working* `GET /graph` *API endpoint and a basic D3.js network visualization. Now you will* ***train a model*** *on this data to predict which companies are most likely to default. First, a simple baseline (logistic regression, no graph structure) to establish a benchmark. Then, a Graph Neural Network that leverages the network topology — this is where the "contagion awareness" enters the model.*

Goal: Train a model that predicts which companies are likely to default, leveraging both financial features AND network structure.

Modeling Pipeline

Step 4A — Baseline Model (Week 4)

> ***Why start with a baseline?*** *Before building complex GNN architectures, you need a simple benchmark to compare against. If a logistic regression using financial features + topological metrics achieves 85% AUC-ROC, then the GNN needs to beat that number to justify its complexity. This baseline also validates your data pipeline — if the simple model can't learn anything useful, the problem is likely in your data, not your model architecture.*

Always build a simple model first — this gives you a working benchmark to beat:

```
# --- Baseline: Logistic Regression (no graph structure) ---
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, roc_auc_score

# Features: financial metrics + topological metrics
feature_cols = ['revenue', 'debt', 'debt_to_equity', 'current_ratio',
                'profit_margin', 'degree', 'betweenness', 'pagerank']
X = df[feature_cols]
y = df['default_label']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
print(f"AUC-ROC: {roc_auc_score(y_test, model.predict_proba(X_test)[:,1]):.4f}")
```

Step 4B — Graph Neural Network (Week 5–6)

Why GNNs? The baseline treats each company independently. But a GNN **aggregates features from neighbors** — so if Supplier B is risky, Company A (its customer) learns to be more cautious. This IS the contagion behavior.

Architecture — Hybrid GNN Stack:

The full model combines multiple GNN components:

Component	Algorithm	What It Does
Message Passing Core	Graph Convolutional Network (GCN)	Isotropic neighborhood aggregation using normalized adjacency matrix
Attention Mechanism	Graph Attention Network (GAT)	Multi-head self-attention → learns which neighbors matter most
Heterophily Handling	CuGNN (Contrastive Unsupervised GNN)	Adaptive low-pass/high-pass filtering for dissimilar connected nodes
Contrastive Learning	Pseudo-Siamese networks	Instance & graph-level contrastive losses to learn cluster-friendly embeddings
Domain Constraints	Physics-Informed GNN (PI-GNN)	Loss penalties enforcing conservation of material flow, capacity limits, lead-time constraints

Mathematical Foundations of the GNN Stack:

GCN — Spectral Graph Convolution (Kipf & Welling, 2017):

The GCN layer-wise propagation rule transforms node features `H^(l)` to `H^(l+1)` using the symmetrically normalized adjacency matrix:

```
Â = Đ^{-1/2} Ã Đ^{-1/2}    where Ã = A + I (adjacency + self-loops), Đ = degree matrix of Ã

H^{(l+1)} = σ(Â · H^{(l)} · W^{(l)})
```

- `H^(l)` = node feature matrix at layer `l` — each row is one company's feature vector
- `W^(l)` = learnable weight matrix at layer `l` — trained via backpropagation
- `σ` = non-linear activation (ReLU) — introduces non-linearity for complex pattern learning
- The `Â` normalization prevents exploding/vanishing features during message passing
- Adding self-loops (`I`) ensures each node retains its own features during aggregation

GAT — Attention Coefficient Computation (Veličković et al., 2018):

GAT computes learned attention weights `α_ij` that determine how much node `j` 's features contribute to node `i` 's updated representation:

```
e_ij = LeakyReLU(aᵀ · [W·h_i || W·h_j])    ← unnormalized attention score
α_ij = softmax_j(e_ij) = exp(e_ij) / Σ_{k∈N(i)} exp(e_ik)    ← normalized attention
coefficient
h'_i = σ(Σ_{j∈N(i)} α_ij · W · h_j)    ← attention-weighted aggregation
```

- `h_i, h_j` = feature vectors of nodes `i` and `j`
- `W` = shared learnable weight matrix that linearly transforms features
- `a` = learnable attention vector that scores neighbor importance
- `||` = concatenation operator
- `N(i)` = neighborhood of node `i`
- Multi-head attention: run `K` independent attention heads and concatenate outputs for stability

Why this matters for credit contagion: GCN treats all neighbors equally (isotropic). GAT learns that Toyota's relationship with its sole-source chip supplier (TSMC) matters *more* than its relationship with a commodity steel vendor. The attention weights `α_ij` directly encode this asymmetric dependency.

Start simple with GraphSAGE (most beginner-friendly GNN):

```python
# --- GraphSAGE GNN for default prediction ---
import torch
import torch.nn.functional as F
from torch_geometric.nn import SAGEConv
from torch_geometric.data import Data

class SupplyChainGNN(torch.nn.Module):
    """
    GraphSAGE-based GNN for credit default prediction.

    How it works for each company node:
      1. Look at its OWN financial data (intrinsic features)
      2. AGGREGATE its suppliers' features (upstream risk)
      3. AGGREGATE its customers' features (downstream risk)
      4. Combine all → predict: default (1) or healthy (0)
    """
    def __init__(self, in_channels, hidden_channels, out_channels):
        super().__init__()
        # Layer 1: aggregate 1-hop neighbors
        self.conv1 = SAGEConv(in_channels, hidden_channels)
        # Layer 2: aggregate 2-hop neighbors (suppliers of suppliers)
        self.conv2 = SAGEConv(hidden_channels, hidden_channels)
        # Final classifier
        self.classifier = torch.nn.Linear(hidden_channels, out_channels)

    def forward(self, x, edge_index):
        # x = node feature matrix [num_nodes, num_features]
        # edge_index = graph connectivity [2, num_edges]

        h = self.conv1(x, edge_index)    # 1st message passing
        h = F.relu(h)                     # non-linear activation
        h = F.dropout(h, p=0.5, training=self.training)

        h = self.conv2(h, edge_index)    # 2nd message passing
```

```
        h = F.relu(h)

        out = self.classifier(h)          # predict default probability
        return F.log_softmax(out, dim=1)  # probability distribution

# Instantiate: 8 input features, 64 hidden units, 2 output classes
model = SupplyChainGNN(in_channels=8, hidden_channels=64, out_channels=2)
```

Preventing Over-smoothing & Over-squashing:

Deep GNNs (many layers) cause all node embeddings to converge — called **representational collapse**. Mitigations:

Problem	Solution	How It Works
Over-smoothing	**DropAGG** (structural aggregation dropout)	Randomly drops neighbor messages during aggregation
Over-squashing	**GRANet** (Graph Random Aggregation)	Samples/discretizes features in high-entropy regions
Gradient vanishing	State-space control formulations	Ensures continuous gradient flow without extra parameters

Evaluation Metrics:

- **Accuracy** — overall correctness
- **F1 Score** — critical for imbalanced data (few defaults vs. many healthy)
- **AUC-ROC** — model's ability to discriminate default vs. healthy
- **Precision/Recall** — false positive vs. false negative tradeoff

Prediction API & Frontend

> ***Closing the ML loop:*** *With the GNN trained, these API tasks expose the model's predictions to the frontend dashboard. The* `POST /predict` *endpoint connects the trained model (from Step 4B above) to the D3.js visualization (from Phase 3). After this step, clicking any company node on the graph will show its GNN-predicted default probability, color-coded by risk tier. This visual output also becomes the "before" snapshot that Phase 5's shock simulation will compare against.*

- [ ] Build `POST /predict` endpoint: accepts company name → returns default probability
- [ ] Display probability on the frontend next to each company node
- [ ] Color-code nodes: 🔴 High risk (PD > 50%), 🟠 Medium (20–50%), 🟢 Low (< 20%)

Phase 5 — Contagion Simulation & Stress Testing (Week 6–7)

> ***Where you are in the pipeline:*** *Phase 4 delivered a trained GNN that outputs a* `PD` *(Probability of Default) for every company, incorporating network-derived risk signals from the graph structure. However, those PDs are* ***static*** *— they reflect the current graph state. The critical next question is: "What happens if a specific company actually defaults?" This phase builds the* ***simulation engine*** *that answers that question. You will inject shocks (removing nodes or edges), re-run the GNN on the perturbed graph, and measure how PDs cascade through the network — exactly the contagion effect described in the introduction. The SIIS epidemic model formalizes this cascade, and the Cox-Copula framework quantifies joint default risk.*

Goal: Simulate what happens when one company suddenly fails and quantify systemic degradation.

Contagion Modeling & Simulation

Task 5A: SIIS Epidemic Contagion Model

Use an extended **SIIS (Susceptible → Infected$_1$ → Infected$_2$ → Susceptible)** model to distinguish:

- **Susceptible (S)** — healthy firm, not currently in distress
- **Infected$_1$ (I_1)** — idiosyncratic distress (internal failure, bad management, operational issues)
- **Infected$_2$ (I_2)** — systemic distress (default caused by upstream/downstream supply chain collapse)

This separation is critical — it lets you measure how much risk is **self-generated** vs. **network-transmitted**.

SIIS State Transitions — Mathematical Formulation:

```
Transition rates for node i at time t:

  S → I₁:  β₁(i)              ← idiosyncratic infection rate (function of node i's financial
health)
  S → I₂:  β₂ · Σ_{j∈N(i)} W_ij · 1_{j ∈ I₁∪I₂}   ← network transmission (weighted sum of
infected neighbors)
  I₁ → S:  γ₁              ← recovery rate from idiosyncratic distress
  I₂ → S:  γ₂              ← recovery rate from systemic distress (γ₂ < γ₁ typically)
  I₁ → I₂:  δ · Σ_{j∈N(i)} W_ij · 1_{j ∈ I₂}   ← escalation: idiosyncratic → systemic when
neighbors also fail
```

- `β₁(i)` depends on firm `i`'s intrinsic features (Altman Z-score, leverage, liquidity)
- `β₂` is the **network contagion rate** — the key parameter we want to estimate
- `W_ij` is the edge weight (trade volume, dependency ratio) — higher dependency = faster transmission
- `1_{j ∈ I₁∪I₂}` is an indicator function: 1 if neighbor `j` is infected, 0 otherwise
- `γ₂ < γ₁` reflects that systemic distress is harder to recover from than idiosyncratic distress
- The **basic reproduction number** `R₀ = β₂ · ⟨k⟩ / γ₂` determines whether contagion spreads ($R_0 > 1$) or dies out ($R_0 < 1$)

Task 5B: Shock Injection Protocols

> ***From theory to practice:*** *The SIIS model above defines how contagion propagates. Now you need to define what triggers it. Shock injection protocols specify the type of disruption you want to simulate — either removing a company entirely (bankruptcy scenario) or severing its connections (trade route disruption). Each shock type reveals different aspects of network vulnerability.*

Two types of shocks to simulate:

Protocol	What It Simulates	How
SCR-NR (Node Removal)	Bankruptcy, disaster, sanctions	Remove or reduce capacity of high-centrality firms
SCR-ER (Edge Removal)	Port closures, strikes, frozen financing	Sever critical supply routes or trade-credit lines

```python
# --- Shock Simulation Engine ---
def simulate_shock(G, model, shocked_company, shock_type='node_removal'):
```

```
    """
    Simulate a credit shock and measure contagion effects.

    Steps:
    1. Record baseline PD for all companies (before shock)
    2. Apply shock (remove node or edges)
    3. Re-run GNN on perturbed graph
    4. Compare before/after PDs
    5. Return ranked list of most-affected companies
    """
    # Step 1: baseline predictions
    baseline_pds = model.predict_all(G)

    # Step 2: apply shock
    if shock_type == 'node_removal':
        # SCR-NR: remove the company entirely
        G_shocked = G.copy()
        G_shocked.remove_node(shocked_company)
    elif shock_type == 'edge_removal':
        # SCR-ER: sever all edges to/from the company
        G_shocked = G.copy()
        edges_to_remove = list(G_shocked.edges(shocked_company))
        G_shocked.remove_edges_from(edges_to_remove)

    # Step 3: re-run GNN on perturbed graph
    shocked_pds = model.predict_all(G_shocked)

    # Step 4: compute delta PDs
    impact = {}
    for company in baseline_pds:
        if company != shocked_company and company in shocked_pds:
            delta = shocked_pds[company] - baseline_pds[company]
            impact[company] = {
                'before': baseline_pds[company],
                'after': shocked_pds[company],
                'delta': delta
            }

    # Step 5: rank by impact
    ranked = sorted(impact.items(), key=lambda x: x[1]['delta'], reverse=True)
    return ranked
```

Example output:

```
Shock: SteelCorp defaults (SCR-NR)

Company          Before PD    After PD    Change
──────────────────────────────────────────────────
Toyota           18%          30%         +12%
Ford             22%          31%         +9%
```

```
ElectroParts     15%          19%          +4%
Honda            25%          27%          +2%
```

Task 5C: Portfolio Risk Recalculation

> ***Quantifying the damage:*** *The shock simulation above gives you qualitative insight ("Toyota's PD increased by 12%"). Now you need to translate those PD changes into financial metrics that a risk manager can act on — how much money is at risk, what's the worst-case loss, and which firms contribute the most to systemic risk.*

After each shock, recalculate:

Metric	Formula	Meaning
Expected Loss (EL)	`EL = PD × LGD × EAD`	Average loss from defaults
Value at Risk (VaR)	95th/99th percentile of loss distribution	Maximum loss at confidence level
Expected Shortfall (ES)	`E[Loss	Loss > VaR]`	
Systemic Risk Contribution (SRC)	Primary losses + cascading secondary losses	Each firm's total systemic footprint

Task 5D: Cox-Copula Joint Default Modeling

Use a **Cox-Copula framework** to evaluate **joint default probabilities** — the chance that multiple companies default simultaneously due to interacting macro and micro factors.

Cox Proportional Hazards Model — models the instantaneous default rate (hazard) for firm `i` at time `t`:

$$h_i(t) = h_0(t) \cdot \exp(\beta_1 \cdot \text{Leverage}_i + \beta_2 \cdot \text{Liquidity}_i + \beta_3 \cdot \text{PageRank}_i + \ldots + \beta_k \cdot x_{ik})$$

- $h_0(t)$ = baseline hazard function (non-parametric — no distributional assumption needed)
- $\exp(\beta^T x_i)$ = multiplicative risk factor based on firm i's covariates (financial + topological features)
- The model is "semi-parametric": it estimates the effect of covariates without specifying the shape of the baseline hazard

Copula Function — models the **dependence structure** between multiple firms' default times:

$$P(T_i \le t, T_j \le t) = C(F_i(t), F_j(t); \theta)$$

- T_i, T_j = default times for firms i and j
- $F_i(t), F_j(t)$ = marginal survival functions (from individual Cox models)
- $C(\cdot, \cdot; \theta)$ = copula function with dependence parameter θ
- Common choices: **Gaussian copula** (linear dependence), **Clayton copula** (lower-tail dependence — good for joint distress), **Gumbel copula** (upper-tail)
- θ captures how strongly defaults are correlated beyond what individual models predict

Why both together: The Cox model gives individual PDs; the Copula captures the *joint* probability that multiple firms default together. In supply chains, joint defaults are far more likely than independent models suggest — precisely because of the contagion channels.

Contagion Simulation & Stress Testing Specification Summary

Simulation Component	Methodology / Protocol	Financial / Network Metrics Output	Strategic Purpose
Epidemic Contagion Modeling	SIIS process with state-dependent transition rates	Transition probabilities between S, I_1, I_2 states	Separate internal risk from network-transmitted systemic risk
Joint Default Probability	Cox-Copula integration with marginal survival functions	Dynamic joint default parameters with tail dependence	Assess joint vulnerabilities on actual transaction paths
Shock Injection Protocol	SCR-NR (node removal) & SCR-ER (edge removal)	Node/edge severing with probability p	Model multi-modal disruptions (geopolitical, cyber, natural)
Risk Amplification Quantification	Network-adjusted PD, EL, VaR, ES recalculation	Recalculated portfolio metrics; amplification factors	Provide true-cost estimates of cascading defaults
Systemic Importance Scoring	Systemic Risk Contribution (SRC) computation	Aggregate primary + contagion-induced secondary losses	Identify firms causing disproportionate systemic risk

Shock Simulator UI

> ***Making simulations accessible:*** *The SIIS engine, shock protocols, risk metrics, and Cox-Copula models above are all backend computations. The Shock Simulator UI wraps these into an interactive interface so that a user can select any company, trigger a simulated default, and instantly see the cascading impact visualized on the network graph and in comparison tables.*

- [ ] Build **Shock Simulator UI**: dropdown to select a company → "Simulate Default" button
- [ ] Show before/after PDs in a comparison table
- [ ] Animate affected nodes on the graph (flash red, increase size proportionally to impact)

Phase 6 — Dashboard & Final Polish (Week 7–8)

> ***Where you are in the pipeline:*** *Phase 5 produced a full contagion simulation engine: SIIS epidemic dynamics, SCR-NR/SCR-ER shock protocols, portfolio risk recalculation (EL, VaR, ES, SRC), and Cox-Copula joint default modeling. You now have everything needed to answer the core question: "If X fails, who else is at risk and by how much?" This final phase wraps all of that analytical power in a* ***user-friendly, interactive dashboard*** *that makes the results accessible to risk managers and decision-makers who don't need to understand graph theory or GNN internals. The dashboard is the* ***delivery vehicle*** *for all the work done in Phases 1–5.*

Goal: Make everything presentable, interactive, and demo-ready.

Risk Dashboard

Build **3 main panels**:

Panel 1: Network Graph (GPU-Accelerated)

- Interactive graph of companies using **GPU-accelerated WebGL force layouts** (e.g., Cosmograph)
- This offloads many-body force computations to GPU for **fluid interaction on large graphs**

- **Node color** = risk level (🔴 🟡 🟢)
- **Node size** = systemic importance (betweenness centrality / SRC)
- Click a node → see its financial details, risk scores, key suppliers/customers

Panel 2: Systemic Supplier Risk Rankings

Company	Systemic Risk Score	Default Probability	SRC Rank
SteelCorp	🔴 High	74%	#1
Toyota	🟡 Medium	31%	#5
ElectroParts	🟢 Low	12%	#12

- Ranked by **Systemic Risk Contribution (SRC)**
- Sortable, filterable, searchable

Panel 3: Shock Scenario Interface

- Dropdown to select a company or edge
- "Simulate Default" / "Sever Route" buttons
- Before/After comparison chart (bar chart or heatmap)
- Contagion radius visualization — animated ripple effect showing how risk propagates
- Predicted downstream rating downgrades and EL amplification overlays

Panel 4 (Stretch): Natural Language Insights

- **NLG (Natural Language Generation)** layer that converts numerical outcomes into actionable recommendations
- Example: "*SteelCorp's default would increase Toyota's PD by 12%. Consider diversifying steel suppliers across 2+ vendors to reduce concentration risk.*"

Data Integration Architecture

- Use **zero-ETL** graph analytics to query data in place across data lakes/warehouses
- Avoids heavy replication; ensures real-time accuracy of customs, financial, and macro streams

API Layer

> ***The bridge between computation and consumption:*** *All the analytical power built in Phases 2–5 (graph data, GNN predictions, shock simulations, risk metrics) needs to be exposed through clean, documented API endpoints. These endpoints allow the React frontend to fetch data on demand and also enable future integration with external systems (ERP platforms, enterprise risk tools).*

- **REST/GraphQL endpoints** for ERP and enterprise risk platform integration:
  - `GET /graph` — full network
  - `GET /companies/{id}` — company details + risk scores
  - `POST /predict` — predict default probability
  - `POST /simulate` — run shock simulation
  - `GET /rankings` — systemic risk rankings

Documentation & Model Cards

- [ ] Document the model architecture, features used, and evaluation results
- [ ] Prepare 3–4 example shock scenarios with screenshots
- [ ] Write the README on GitHub with setup instructions

- [ ] Create model cards explaining training data, hyperparameters, and performance

Dashboard Engineering Specification Summary

Infrastructure / UI Component	Technology / Technique	Functional Capability	Business Value
Large-Scale Rendering Engine	GPU-accelerated WebGL force layouts (Cosmograph)	Offload many-body force sims to GPU for real-time rendering	Interactive exploration across multi-million node graphs
Data Integration Architecture	Zero-ETL graph analytics	Direct query translation over existing lakes/warehouses	Real-time metrics; reduced maintenance & replication
Scenario Command Interface	Interactive UI/UX with React + D3.js	Click/remove nodes/edges and trigger backend GNN recalcs	Democratize stress-testing for non-technical users
Insight Translation Layer	Natural Language Generation (NLG)	Convert metrics into plain-language alerts & recommendations	Bridge deep learning outputs to procurement decisions
Enterprise Integration	REST/GraphQL APIs & microservices	Expose GNN inference & scenario endpoints	Embed contagion awareness into ERP / credit pipelines

---

⚠️ Risk Assessment & Mitigations

Over-smoothing & Over-squashing

- **Problem:** Deep message passing (many GNN layers) causes all node embeddings to converge toward the same vector — called **representational collapse**. Information from distant nodes gets "squashed" through bottleneck paths, losing discriminative signal.
- **Technical detail:** After L layers of message passing, each node's embedding contains information from its L-hop neighborhood. For large L, all nodes see essentially the same global average, destroying local structure.
- **Fix:** DropAGG (structural aggregation dropout — randomly drops neighbor messages during training); GRANet (samples/discretizes features in high-entropy regions via SDE methods); state-space control formulations to ensure continuous gradient flow without excessive trainable parameters.

Data Sparsity in Deep Tiers

- **Problem:** Visibility into supply chains drops sharply after tier-1. Tier-2 and tier-3 suppliers are often private companies with limited financial data. Sparse bridges between subgraphs can skew GAT attention weights and cause false negatives (missing real dependencies).
- **Fix:** CuGNN contrastive learning to infer latent (unobserved) dependencies from structural patterns; PI-GNN physics-informed bounds to prevent the model from hallucinating supply flows or impossible production capacities.

Heterophily

- **Problem:** Financial supply-chain networks frequently connect **dissimilar** nodes (e.g., a tiny $50M-revenue fastener supplier to a $280B-revenue OEM like Toyota). Standard homophilic GNNs (designed for networks

where connected nodes are similar) fail in this setting because message passing assumes neighbor features should be averaged together.
- **Fix:** CuGNN adaptive filters that balance low-pass filtering (smoothing for similar neighbors) with high-pass filtering (preserving signal for dissimilar neighbors) + specialized contrastive objectives that learn meaningful embeddings even for disparate node pairs.

Class Imbalance in Default Prediction

- **Problem:** Actual corporate defaults are rare events (< 2% annual default rate). With ~300 companies, you may have fewer than 10 positive labels, causing the GNN to predict "healthy" for everything.
- **Fix:** SMOTE oversampling on graph embeddings; focal loss instead of cross-entropy; stratified train/test splits; use Altman Z-Score thresholds to create a more balanced proxy label distribution.

✅ Final Deliverables Checklist

- [ ] GitHub repo with clean, commented code
- [ ] Supply chain dataset (CSV) with ~300 companies, ~1,000 relationships
- [ ] Unified master entity index with UUIDs
- [ ] Feature matrices: intrinsic (financial) + extrinsic (topological) + temporal
- [ ] Baseline ML model (Logistic Regression) with evaluation metrics
- [ ] Trained hybrid GNN (GCN + GAT + CuGNN + PI-GNN) with contrastive pretraining
- [ ] SIIS contagion simulation engine with SCR-NR / SCR-ER shock protocols
- [ ] Portfolio risk metrics: EL, VaR, ES, SRC
- [ ] GPU-accelerated interactive risk dashboard (React + WebGL)
- [ ] Network graph visualization with color-coded risk levels
- [ ] REST/GraphQL API endpoints
- [ ] Project README with setup instructions
- [ ] Demo video or live deployment

🛠️ Implementation Notes & Next Steps

A technical checklist of infrastructure and engineering decisions to finalize during implementation:

- [ ] Finalize master entity resolution pipeline and UUID schema
- [ ] Choose graph database technology for production (scale, property support, query latency) — candidates: Neo4j, Memgraph, or keep NetworkX for prototype
- [ ] Build HTML → Markdown parser templates for regulatory filings and design NER annotation schema for SEC 10-K extraction
- [ ] Instrument tCNN pipelines for irregular time series and validate temporal encodings on Panjiva or proxy data
- [ ] Prototype GCN + GAT stack; integrate CuGNN contrastive pretraining and PI-GNN domain constraints
- [ ] Implement DropAGG / GRANet regularization modules and test for over-smoothing on synthetic deep-tier graphs
- [ ] Develop SCR-NR / SCR-ER simulation harness and validate amplification multipliers against historical case studies (e.g., 2011 Fukushima, 2021 chip shortage, 2024 Red Sea disruption)
- [ ] Implement GPU WebGL front-end (Cosmograph) prototype and zero-ETL query adapters

- [ ] Define governance, explainability, and regulatory reporting requirements for model output consumption
- [ ] Set up CI/CD pipeline for automated testing and model retraining

---

📚 Learning Resources

Topic	Resource
Python basics	Python for Everybody (free, Coursera)
Pandas & NumPy	Kaggle's free micro-courses
NetworkX	Official NetworkX Tutorial
GNN fundamentals	CS224W: ML with Graphs (Stanford, free YouTube)
PyTorch Geometric	Official PyG tutorials
NLP/NER	spaCy 101
FastAPI	FastAPI official tutorial
React basics	React official tutorial
D3.js graph viz	D3 Graph Gallery
WebGL rendering	Cosmograph docs
Git & GitHub	GitHub's beginner guide
Credit Risk Modeling	Moody's Analytics Credit Risk Resources
Graph Databases	Neo4j GraphAcademy (free)
Survival Analysis	Lifelines Python Library Docs

Appendix: Notation & Acronyms

Symbol	Meaning
`G = (V, E, X, W)`	Property graph (nodes, edges, node features, edge weights)
`A`	Adjacency matrix of the graph
`Â`	Symmetrically normalized adjacency `Đ^{-1/2} Ã Đ^{-1/2}`
`D`	Degree matrix (diagonal)
`L`	Graph Laplacian `L = D - A`
`H^(l)`	Node feature matrix at GNN layer `l`
`W^(l)`	Learnable weight matrix at GNN layer `l`
`α_ij`	GAT attention coefficient (node `j`'s importance to node `i`)

PD	Probability of Default
EL	Expected Loss (PD × LGD × EAD)
VaR	Value at Risk
ES	Expected Shortfall
SRC	Systemic Risk Contribution
h_i(t)	Hazard function for firm i at time t (Cox model)
C(·, ·; θ)	Copula function with dependence parameter θ
β_1, β_2	SIIS idiosyncratic and network contagion rates
γ_1, γ_2	SIIS recovery rates (idiosyncratic and systemic)
R_0	Basic reproduction number ($\beta_2 \cdot \langle k \rangle / \gamma_2$)
SIIS	Extended Susceptible-Infected$_1$-Infected$_2$-Susceptible epidemic process
GCN	Graph Convolutional Network
GAT	Graph Attention Network
CuGNN	Contrastive Unsupervised GNN
PI-GNN	Physics-Informed GNN
tCNN	temporal Convolutional Neural Network
DropAGG	Structural aggregation dropout mechanism
GRANet	Graph Random Aggregation Network
SCR-NR	Supply Chain Resilience — Node Removal
SCR-ER	Supply Chain Resilience — Edge Removal
LGD	Loss Given Default
EAD	Exposure at Default
COGS	Cost of Goods Sold

> 💬 ***The Entire Project in One Sentence:*** *Build a heterogeneous supply-chain graph of ~300 companies, train a hybrid GNN (GCN + GAT + CuGNN + PI-GNN) to predict network-adjusted default probabilities, simulate credit contagion via SIIS epidemic models and SCR-NR/SCR-ER shock protocols, and deliver a GPU-accelerated interactive dashboard with systemic risk rankings, shock simulation, and NLG-powered insights for risk managers.*